

Spring Boot — Circular Dependency, Bean Scopes & Initialization

Chapter Flow

1. Circular Dependency — Introduction
 2. Circular Dependency in Core Java
 3. Field & Setter Injection
 4. Why Circular Dependency Should Be Avoided
 5. Removing Circular Dependency
 6. Bean Scopes
 7. Singleton Scope
 8. Prototype Scope
 9. Singleton Confusion with `@Bean`
 10. When to Use Which Scope
 11. Other Scopes
 12. Bean Initialization — Eager
 13. `@Lazy` & Lazy Initialization
 14. Prototype Scope is Lazy
 15. Why Default Initialization is Eager
 16. Solving Circular Dependency Using `@Lazy`
-

1. Circular Dependency

Dependency

If class `A` requires an object of class `B`, then:

`A → B`

`A` is dependent on `B`.

Example:

```
class OrderService {  
    private PaymentService paymentService;  
}
```

Here:

```
OrderService → PaymentService
```

2. What is Circular Dependency?

A **Circular Dependency** occurs when dependencies form a cycle.

Direct Circular Dependency



Meaning:

```
A depends on B
B depends on A
```

Example:

```
OrderService → PaymentService
PaymentService → OrderService
```

The dependency graph becomes:

```
OrderService
  ↓
PaymentService
  ↓
OrderService
  ↓
PaymentService
  ↓
...
```

3. Circular Dependency in Core Java

Circular Dependency is **not specific to Spring**.

It can happen in normal Java code.

Example:

```
class A {  
  
    B b;  
  
    A() {  
        b = new B();  
    }  
}
```

```
class B {  
  
    A a;  
  
    B() {  
        a = new A();  
    }  
}
```

Now:

```
new A();
```

causes:

```
new A()  
  ↓  
A constructor  
  ↓  
new B()  
  ↓  
B constructor  
  ↓  
new A()  
  ↓  
A constructor  
  ↓  
new B()
```

↓
...

The constructors continuously create each other.

This eventually results in a `StackOverflowError` because the call stack keeps growing.

Key Point

Circular Dependency
≠
Spring-specific problem

It is fundamentally a **dependency/design problem**.

4. Circular Dependency with Constructor Injection

Suppose:

```
@Component
class A {

    private B b;

    A(B b) {
        this.b = b;
    }
}
```

and:

```
@Component
class B {

    private A a;

    B(A a) {
        this.a = a;
    }
}
```

Dependency graph:



To create **A** :

A requires B

To create **B** :

B requires A

Therefore:

```
Create A
  ↓
Need B
  ↓
Create B
  ↓
Need A
  ↓
Cycle
```

There is no dependency that can be created first through constructor injection.

5. Field Injection / Setter Injection

The chapter demonstrates that **field injection and setter injection can allow Spring to handle certain circular dependencies**, because the dependency does not have to be supplied as part of the constructor call.

Field Injection

```
@Component
class A {
```

```
@Autowired
private B b;
}
```

Setter Injection

```
@Component
class A {

    private B b;

    @Autowired
    public void setB(B b) {
        this.b = b;
    }
}
```

Important Difference

Constructor injection:

```
Create object
    ↓
Dependency required immediately
```

Field/Setter injection:

```
Create object
    ↓
Dependency injected afterward
```

This gives Spring an opportunity to create the objects and then inject the dependencies.

6. Should We Solve Circular Dependency?

Recommended approach:

Avoid Circular Dependency instead of trying to solve it.

If:

```
A → B
B → A
```

ask:

Why does `A` need `B`, and why does `B` need `A`?

Usually the better solution is to **refactor the design**.

7. Removing Circular Dependency

Instead of:

```
A ↔ B
```

try to redesign responsibilities so that dependency flows in one direction:

```
A → B
```

or introduce another component if appropriate:

```
A → C ← B
```

The goal is to remove the cycle rather than make the cycle easier for Spring to manage.

Important

```
@Lazy
Field Injection
Setter Injection
```

can help technically with dependency creation, but they should **not be the first choice for fixing a bad dependency design**.

8. Bean Scope

A **Bean Scope** defines the **lifecycle/instance-creation behavior of a Spring Bean**.

The two important scopes in this chapter are:

1. Singleton
2. Prototype

9. Bean Definition vs Bean Instance

This distinction is extremely important.

Bean Definition

A **Bean definition** is Spring's metadata/instructions describing a Bean, such as:

- Bean type
- How to create it
- Scope
- Configuration

Example:

```
@Component
class PaymentService {
}
```

This tells Spring to manage a Bean of type `PaymentService`.

Bean Instance

The **Bean instance** is the actual Java object created from that Bean definition.

```
Bean Definition
  ↓
Java Object
  ↓
Bean Instance
```

Remember

Bean Definition $\neq$ Bean Instance

10. Singleton Scope

Precise Definition

Singleton scope means Spring creates exactly one Bean instance for a particular Bean definition within a particular Spring IoC container.

This is the precise meaning.

It does **NOT** mean:

There can only be one object of that Java class in the entire application/JVM.

11. Singleton — One Instance Per Bean Definition

Suppose:

```
@Component
class PaymentService {
}
```

Spring has a Bean definition:

```
Bean Definition
  |
  | scope = Singleton
  ↓
One Bean Instance
```

If you request that same Bean multiple times:

```
context.getBean(PaymentService.class);
context.getBean(PaymentService.class);
context.getBean(PaymentService.class);
```

Spring returns the **same Bean instance**.

```
getBean() ┌
getBean() ├──→ PaymentService Object #1
getBean() └
```

Therefore:

```
p1 == p2
```

is:

```
true
```

12. "One Object Per Bean Definition" — Why This Matters

Consider:

```
@Component
class PaymentService {
}
```

There is one Bean definition:

```
PaymentService Bean Definition
      ↓
      Object #1
```

But imagine you explicitly define **two different Bean definitions** that create the same class:

```
@Bean
PaymentService paymentService1() {
    return new PaymentService();
}

@Bean
PaymentService paymentService2() {
```

```
    return new PaymentService();  
}
```

Now there are **two Bean definitions**:

```
paymentService1 Bean Definition  
    ↓  
    PaymentService #1  
  
paymentService2 Bean Definition  
    ↓  
    PaymentService #2
```

Even though both have Singleton scope, they are **two different Singleton Bean instances**, because Singleton applies **per Bean definition**.

Therefore:

```
Singleton  
=  
One instance per Bean definition per IoC container
```

NOT:

```
Singleton  
=  
One instance per Java class
```

This is one of the most important distinctions in this chapter.

13. Singleton is Container-Scoped

Singleton is also relative to the **Spring IoC container**.

Conceptually:

```
ApplicationContext A  
    ↓  
PaymentService Bean
```

↓
Object #1

Another independent container could have:

ApplicationContext B
↓
PaymentService Bean
↓
Object #2

So Singleton does not mean one object across all possible Spring containers.

Precise Definition

One Bean instance
↓
for each Bean definition
↓
within one Spring IoC container

14. Singleton Example

```
@Component  
public class PaymentService {  
}
```

Then:

```
PaymentService p1 =  
    context.getBean(PaymentService.class);  
  
PaymentService p2 =  
    context.getBean(PaymentService.class);
```

Because the Bean definition uses the default Singleton scope:

p1 ————
↓

```
PaymentService #1
      ↑
p2 ———
```

Therefore:

```
p1 == p2
```

is:

```
true
```

15. Prototype Scope

Precise Definition

Prototype scope means Spring creates a new Bean instance each time the Bean is requested from the container.

Example:

```
@Component
@Scope("prototype")
public class PaymentService {
}
```

Now:

```
PaymentService p1 =
    context.getBean(PaymentService.class);

PaymentService p2 =
    context.getBean(PaymentService.class);
```

Spring creates:

p1 → PaymentService Object #1

p2 → PaymentService Object #2

Therefore:

p1 == p2

is:

false

16. Singleton vs Prototype

Feature	Singleton	Prototype
Instances	One per Bean definition per IoC container	New instance per Bean request
Default scope?	Yes	No
Same object returned repeatedly?	Yes	No
Object creation	One managed instance	New instance each request
Initialization behavior discussed	Eager by default	Lazy by nature

Memory Trick

Singleton
→ Same instance

Prototype
→ New instance

17. @Bean and Singleton Confusion

Consider:

```
@Configuration
class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

You may think:

```
@Bean method
    ↓
new PaymentService()
    ↓
Every method call = new object?
```

That is not how you should think about the managed Bean.

The returned object is registered as a Spring Bean, and by default its scope is **Singleton**.

Therefore the Bean is managed as:

```
paymentService Bean Definition
    ↓
One Singleton Bean Instance
```

18. Important @Bean Rule

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

By default:

Scope = Singleton

The `new PaymentService()` expression creates the object used for that Bean definition; it does **not** mean that every retrieval of the Spring Bean creates another instance.

Remember

```
@Bean
+
Default Scope
  ↓
Singleton
```

19. When to Use Singleton?

Use Singleton when you want:

One shared Bean instance

It is particularly suitable for **stateless services/components**, where sharing one instance does not create unwanted shared mutable state.

Example conceptually:

```
OrderService
PaymentService
EmailService
```

if their instances do not maintain request-specific mutable state.

20. When to Use Prototype?

Use Prototype when you need:

A separate Bean instance for each request to the container

It can be appropriate when the Bean maintains state that should not be shared between separate Bean requests.

Example:

```
Request 1 → Object #1  
Request 2 → Object #2  
Request 3 → Object #3
```

21. Other Bean Scopes

The chapter briefly introduces additional scopes.

Common Spring scopes include:

```
Singleton  
Prototype  
Request  
Session  
Application  
WebSocket
```

The detailed focus of this chapter is:

```
Singleton  
Prototype
```

The others are introduced rather than covered in the same depth.

22. Scope vs Initialization

These are **two different concepts**.

Scope asks:

How does Spring manage Bean instances?

Examples:

Singleton
Prototype

Initialization asks:

When is the Bean instance created?

Examples:

Eager
Lazy

Therefore, don't confuse:

Singleton

with:

Eager

They describe different things.

23. Eager Initialization

For Singleton Beans, the default initialization behavior is **Eager**.

Eager means:

Spring creates the Singleton Bean during application/context startup rather than waiting for the Bean to be explicitly requested.

Flow:

```
Application starts
    ↓
Spring Container starts
    ↓
Singleton Bean created
```

↓
Application continues

24. Why Eager Initialization?

One major benefit is **early detection of configuration/dependency problems**.

Suppose a Bean has an invalid dependency.

With eager initialization:

```
Application startup
  ↓
Create Bean
  ↓
Resolve dependencies
  ↓
Problem detected
  ↓
Application startup fails
```

The problem is discovered immediately.

25. Lazy Initialization

Lazy initialization means:

Spring delays Bean creation until the Bean is actually needed.

Example:

```
@Component
@Lazy
public class PaymentService {
}
```

Flow:

```
Application starts
  ↓
PaymentService NOT created
  ↓
Later PaymentService is requested
  ↓
PaymentService created
```

Memory Trick

```
Eager
→ Create early

Lazy
→ Create when needed
```

26. @Lazy

`@Lazy` tells Spring to initialize the applicable Bean lazily.

Example:

```
@Component
@Lazy
public class PaymentService {
}
```

Without `@Lazy`:

```
Startup → Bean created
```

With `@Lazy`:

```
Startup → Bean not created

First required use
  ↓
Bean created
```

27. Prototype Scope is Lazy

Prototype Beans are created when they are requested.

Example:

```
@Component
@Scope("prototype")
class PaymentService {
}
```

At startup:

No Prototype instance needs to be created

When requested:

```
getBean()
↓
Create new instance
```

Next request:

```
getBean()
↓
Create another new instance
```

Therefore:

```
Prototype
→ New instance when requested
→ Naturally lazy in creation
```

28. Singleton + Eager vs Singleton + Lazy

Singleton and Lazy are not opposites.

A Singleton Bean can be:

Singleton + Eager

```
One instance
+
Created during startup
```

Singleton + Lazy

```
One instance
+
Created when first needed
```

The **number/lifetime behavior** comes from the scope.

The **creation timing** comes from initialization.

29. Default Behavior

For the concepts covered in this chapter:

```
Singleton
  ↓
One instance per Bean definition per IoC container
  ↓
Eager initialization by default
```

Prototype:

```
Prototype
  ↓
New instance for each request
  ↓
Created when requested
```

30. Why is Eager the Default?

The main reason emphasized in the chapter is **early failure detection**.

With Eager initialization:

```
Application startup
    ↓
Create Singleton Beans
    ↓
Resolve dependencies
    ↓
Detect configuration/dependency problems
    ↓
Fail early if something is wrong
```

With Lazy initialization:

```
Application startup
    ↓
Bean not created
    ↓
Application may start successfully
    ↓
Bean requested later
    ↓
Creation/configuration problem discovered later
```

Therefore, eager initialization helps expose problems **during startup** instead of potentially much later.

31. Global Lazy Initialization

Spring Boot provides:

```
spring.main.lazy-initialization=true
```

This enables lazy initialization globally for applicable Beans.

Conceptually:

Default:
Singleton Beans → Eager

Global Lazy:
Applicable Beans → Lazy

32. Default Value

The default value is:

```
spring.main.lazy-initialization=false
```

Therefore, unless global lazy initialization is enabled, the normal eager initialization behavior remains.

33. @Lazy(false)

If global lazy initialization has been enabled:

```
spring.main.lazy-initialization=true
```

but one particular Bean should be initialized eagerly, use:

```
@Lazy(false)
```

Example:

```
@Component
@Lazy(false)
public class PaymentService {
}
```

Conceptually:

```
Global configuration
↓
```

```
Everything applicable = Lazy
    ↓
PaymentService
    ↓
@Lazy(false)
    ↓
PaymentService = Eager
```

34. Solving Circular Dependency with @Lazy

Consider:

```
A → B
B → A
```

with constructor injection.

Without Lazy:

```
Create A
    ↓
Need B
    ↓
Create B
    ↓
Need A
    ↓
Circular Dependency
```

35. Using @Lazy on the Dependency

Example:

```
@Component
class A {

    private final B b;

    A(@Lazy B b) {
```

```
        this.b = b;
    }
}
```

The important point is:

```
A requires B
  ↓
B is Lazy
  ↓
Don't immediately create real B
  ↓
Inject a proxy/reference
```

This allows Spring to finish creating `A`.

36. What is the Proxy Doing?

Conceptually:

```
A
↓
B dependency
↓
B Proxy
```

The proxy acts as a stand-in for the actual `B` object.

The real `B` Bean can be created later when it is actually needed.

37. Circular Dependency with `@Lazy` — Flow

Suppose:

```
A → B
B → A
```

and `B` is Lazy.

Step 1

Spring starts creating:

A

Step 2

A requires B.

Step 3

B is Lazy.

Therefore:

```
Real B
  ↓
not created immediately
```

Instead:

```
B Proxy
  ↓
injected into A
```

Step 4

A is now created successfully.

Step 5

Later, when the application actually needs B :

```
B Proxy
  ↓
Real B created
  ↓
B receives A
```

Thus the immediate creation cycle is avoided.

38. Why @Lazy Can Break the Cycle

Without Lazy:

```
A creation
↓
B creation
↓
A creation
↓
Cycle
```

With Lazy:

```
A creation
↓
B required
↓
B is Lazy
↓
Proxy injected
↓
A creation completes
↓
Later B required
↓
Real B created
```

The difference is **when the dependency is actually created**.

39. Important Limitation of @Lazy

@Lazy does **not** mean:

Circular Dependency is good design.

It only changes Bean creation timing and can allow Spring to resolve the immediate creation cycle.

The preferred design is still:

Remove the Circular Dependency

rather than:

Keep the Circular Dependency
+
Add @Lazy

40. Most Important Definitions

Dependency

A → B

A requires B.

Circular Dependency

A → B
B → A

Dependencies form a cycle.

Bean Definition

Spring's metadata/instructions describing a managed Bean.

Bean Instance

The actual Java object created from a Bean definition.

Singleton

Exactly one Bean instance
per Bean definition
per Spring IoC container

Prototype

New Bean instance
for each request to the container

Eager

Bean created during startup

Lazy

Bean creation delayed until needed

@Lazy

Makes applicable Bean/dependency lazy

41. Most Important Interview Distinctions

Singleton ≠ One Object Per Class

✗ Incorrect:

Singleton = only one object of a class

✓ Correct:

Singleton =
one Bean instance per Bean definition
per Spring IoC container

Scope ≠ Initialization

✗ Incorrect:

Singleton means Eager

More precisely:

Singleton = instance/scoping behavior
Eager = initialization timing

A Singleton Bean can also be Lazy.

Prototype ≠ Singleton

Singleton:
Same managed instance returned

Prototype:
New instance created for each Bean request

Circular Dependency ≠ Spring-only Problem

It can occur in:

Plain Java
+
Spring

Spring provides mechanisms that can sometimes handle it, but the underlying problem is dependency design.

42. Final Revision Table

Concept	Precise Meaning
Dependency	One component requires another component
Circular Dependency	Dependencies form a cycle, e.g. <code>A → B → A</code>
Constructor Injection	Dependency must be supplied during object construction

Concept	Precise Meaning
Field Injection	Dependency is injected into a field after object construction
Setter Injection	Dependency is supplied through a setter
Bean Definition	Metadata/instructions describing a Spring-managed Bean
Bean Instance	Actual Java object created from the Bean definition
Singleton	Exactly one instance per Bean definition per IoC container
Prototype	New instance for each request to the container
Eager	Bean created during startup
Lazy	Bean creation delayed until required
<code>@Lazy</code>	Enables lazy initialization for the applicable Bean/dependency
<code>@Lazy(false)</code>	Disables lazy initialization for that Bean
Global Lazy	<code>spring.main.lazy-initialization=true</code>
Proxy	Stand-in used for a lazy dependency until the actual Bean is needed

43. One-Minute Revision

CIRCULAR DEPENDENCY

A → B → A

↓

Avoid / refactor it

↓

Field/Setter injection can sometimes allow Spring to resolve it

↓

`@Lazy` can also break the immediate creation cycle

BEAN SCOPE

↓

Singleton

→ One instance per Bean definition
per IoC container

Prototype

→ New instance for every Bean request

INITIALIZATION

↓
Eager
→ Create during startup

Lazy
→ Create when needed

DEFAULT
↓
Singleton
+
Eager

GLOBAL LAZY
↓
spring.main.lazy-initialization=true

CIRCULAR DEPENDENCY + @Lazy
↓
Lazy dependency
↓
Proxy
↓
First Bean can be created
↓
Real dependency created later

★ Golden Rules

1. **Singleton** = one instance per Bean definition per Spring IoC container.
2. **Singleton** does NOT mean one object per Java class.
3. **Two Singleton Bean** definitions can produce two different instances of the same class.
4. **Prototype** = new Bean instance for every request to the container.
5. **Scope** determines instance-creation/lifecycle behavior; **initialization** determines when creation happens.
6. **Singleton** is eager by default.
7. **Prototype** is created when requested.
8. `@Lazy` delays Bean creation.
9. **Eager initialization** helps detect Bean/dependency problems early during startup.
10. **Circular Dependency** should preferably be removed through refactoring.
11. **Field/Setter injection** can allow certain circular dependencies because dependencies can be injected after construction.

12. `@Lazy` can break a circular creation cycle by delaying one dependency and using a proxy.
13. `@Lazy` is a technical workaround, not a replacement for good dependency design.